

Listing 8.3 Implementing `forAll`

A stream of pairs `(a, i)` where `a` is a random value and `i` is its index in the stream.

If a test case generates an exception, record it in the result.

Generates an infinite stream of `A` values by repeatedly sampling a generator.

```
def forAll[A](as: Gen[A])(f: A => Boolean): Prop = Prop {
  (n, rng) => randomStream(as)(rng).zip(Stream.from(0)).take(n).map {
    case (a, i) => try {
      if (f(a)) Passed else Falsified(a.toString, i)
    } catch { case e: Exception => Falsified(buildMsg(a, e), i) }
  }.find(_.isFalsified).getOrElse(Passed)
}
```

When a test fails, record the failed case and its index so we know how many tests succeeded before it.

```
def randomStream[A](g: Gen[A])(rng: RNG): Stream[A] =
  Stream.unfold(rng)(rng => Some(g.sample.run(rng)))
def buildMsg[A](s: A, e: Exception): String =
  s"test case: $s\n" +
  s"generated an exception: ${e.getMessage}\n" +
  s"stack trace:\n ${e.getStackTrace.mkString("\n")}"
```

String interpolation syntax. A string starting with `s"` can refer to a Scala value `v` as `$v` or `${v}` in the string. The Scala compiler will expand this to `v.toString`.

Notice that we're catching exceptions and reporting them as test failures, rather than letting the run throw the error (which would lose information about what argument triggered the failure).



EXERCISE 8.9

Now that we have a representation of `Prop`, implement `&&` and `||` for composing `Prop` values. Notice that in the case of failure we don't know which property was responsible, the left or the right. Can you devise a way of handling this, perhaps by allowing `Prop` values to be assigned a tag or label which gets displayed in the event of a failure?

```
def &&(p: Prop): Prop
def ||(p: Prop): Prop
```

8.3 Test case minimization

Earlier, we mentioned the idea of test case minimization. That is, ideally we'd like our framework to find the smallest or simplest failing test case, to better illustrate the problem and facilitate debugging. Let's see if we can tweak our representations to support this outcome. There are two general approaches we could take:

- *Shrinking*—After we've found a failing test case, we can run a separate procedure to minimize the test case by successively decreasing its "size" until it no longer fails. This is called *shrinking*, and it usually requires us to write separate code for each data type to implement this minimization process.

- *Sized generation*—Rather than shrinking test cases after the fact, we simply generate our test cases in order of increasing size and complexity. So we start small and increase the size until we find a failure. This idea can be extended in various ways to allow the test runner to make larger jumps in the space of possible sizes while still making it possible to find the smallest failing test.

ScalaCheck, incidentally, takes the first approach: shrinking. There’s nothing wrong with this approach (it’s also used by the Haskell library QuickCheck that ScalaCheck is based on: <http://mng.bz/E24n>), but we’ll see what we can do with sized generation. It’s a bit simpler and in some ways more modular, because our generators only need to know how to generate a test case of a given size. They don’t need to be aware of the “schedule” used to search the space of test cases, and the function that runs the tests therefore has the freedom to choose this schedule. We’ll see how this plays out shortly.

Instead of modifying our `Gen` data type, for which we’ve already written a number of useful combinators, let’s introduce sized generation as a separate layer in our library. A simple representation of a sized generator is just a function that takes a size and produces a generator:

```
case class SGen[+A](forSize: Int => Gen[A])
```



EXERCISE 8.10

Implement helper functions for converting `Gen` to `SGen`. You can add this as a method on `Gen`.

```
def unsized: SGen[A]
```



EXERCISE 8.11

Not surprisingly, `SGen` at a minimum supports many of the same operations as `Gen`, and the implementations are rather mechanical. Define some convenience functions on `SGen` that simply delegate to the corresponding functions on `Gen`.⁵



EXERCISE 8.12

Implement a `listOf` combinator that doesn’t accept an explicit size. It should return an `SGen` instead of a `Gen`. The implementation should generate lists of the requested size.

```
def listOf[A](g: Gen[A]): SGen[List[A]]
```

⁵ In part 3 we’ll discuss ways of factoring out this sort of duplication.